

ใบความรู้ที่ 6 การเรียนรู้เชิงเสริมแรง

วัตถุประสงค์การเรียนรู้

1. รู้ความหมายและปรัชญาพื้นฐานของการเรียนรู้เชิงเสริมแรง (Reinforcement Learning)
2. เข้าใจความแตกต่างระหว่างการเรียนรู้แบบมีผู้สอน (Supervised), ไม่มีผู้สอน (Unsupervised) และเชิงเสริมแรง (Reinforcement)
3. รู้คำศัพท์เทคนิคและองค์ประกอบสำคัญของระบบ (Agent, Environment, State, Action, Reward)
4. เข้าใจกระบวนการตัดสินใจแบบมาร์คอฟ (MDP)
5. เข้าใจคอนเซปต์ของสมการเบลล์แมน (Bellman Equation) ในฐานะเครื่องมือที่ช่วยให้ AI มองเห็นอนาคตผ่านการเปรียบเทียบกับสถานการณ์จริง
6. เข้าใจหลักการทำงานของอัลกอริทึม Q-Learning และ Epsilon-Greedy ผ่านโครงสร้างภาษา Python พื้นฐานได้
7. รู้วิธีการประยุกต์ใช้ทฤษฎีเพื่อสร้างปัญญาประดิษฐ์สำหรับเล่นเกม Tic-Tac-Toe

1. ปรัชญาและรากฐานทฤษฎีของการเรียนรู้เชิงเสริมแรง

ในโลกของปัญญาประดิษฐ์ (Artificial Intelligence) การเรียนรู้เชิงเสริมแรง หรือ Reinforcement Learning (RL) มิใช่เพียงแค่อัลกอริทึมทางคณิตศาสตร์ชุดหนึ่ง แต่เป็น "กระบวนทัศน์" (Paradigm) ที่สะท้อนวิธีคิดเกี่ยวกับการเรียนรู้ของสิ่งมีชีวิตในธรรมชาติอย่างลึกซึ้งที่สุด หากเราเปรียบเทียบกับมนุษย์ การเรียนรู้แบบมีผู้สอน (Supervised Learning) เปรียบเสมือนการเรียนรู้ในห้องเรียนที่มีครูคอยเฉลยคำตอบที่ถูกต้องให้ตลอดเวลา (Flashcards) และการเรียนรู้แบบไม่มีผู้สอน (Unsupervised Learning) เปรียบเสมือนการนั่งสมาธิเพื่อจัดระเบียบความคิดและค้นหาโครงสร้างในสมองตนเอง แต่ Reinforcement Learning คือการเรียนรู้จากการใช้ชีวิตจริง (Learning from Life)

1.1. จากพฤติกรรมนิยมสู่วิทยาการคำนวณ (Behaviorism to Computational Intelligence)

รากฐานทางปรัชญาของ RL เชื่อมโยงโดยตรงกับทฤษฎีจิตวิทยาพฤติกรรมนิยม (Behaviorism) ของ B.F. Skinner และ Ivan Pavlov ซึ่งอธิบายว่าสิ่งมีชีวิตเรียนรู้ที่จะแสดงพฤติกรรมผ่านกลไกการเสริมแรง (Reinforcement Mechanism)

1.1.1. การเสริมแรงทางบวก (Positive Reinforcement): เมื่อการกระทำได้นำมาซึ่งความพึงพอใจ (Pleasure) หรือรางวัล (Reward) สมองจะหลั่งสารสื่อประสาทโดปามีน (Dopamine) เพื่อกระตุ้นให้เกิดการทำซ้ำพฤติกรรมนั้น

1.1.2. การเสริมแรงทางลบ (Negative Reinforcement/Punishment): เมื่อการกระทำนำมาซึ่งความเจ็บปวด (Pain) หรือผลลัพธ์ที่ไม่พึงประสงค์ สมองจะจดจำเพื่อหลีกเลี่ยงสถานการณ์นั้นในอนาคต

ในบริบทของวิทยาการคอมพิวเตอร์ เราแปลงนามธรรมทางจิตวิทยานี้ให้เป็นสมการทางคณิตศาสตร์ โดยสร้าง "ตัวแทน" (Agent) ที่มีเป้าหมายสูงสุดเพียงอย่างเดียวคือ การทำให้ผลรวมของรางวัลสะสมมีค่ามากที่สุด (Maximize Cumulative Reward) ภายใต้สภาพแวดล้อมที่มีความไม่แน่นอน (Stochastic Environment)

1.2. ความแตกต่างเชิงโครงสร้างของกระบวนทัศน์การเรียนรู้ (Paradigm Comparison)

มิติการเปรียบเทียบ (DIMENSION)	SUPERVISED LEARNING (การเรียนรู้ แบบมีผู้สอน)	UNSUPERVISED LEARNING (การเรียนรู้ แบบไม่มีผู้สอน)	REINFORCEMENT LEARNING (การเรียนรู้ เชิงเสริมแรง)
บทบาทของข้อมูล (DATA ROLE)	เรียนรู้จากข้อมูลดีที่มี ฉลาก (Labeled Data) ที่ถูกระบุไว้แล้ว	เรียนรู้จากข้อมูลดิบ (Unlabeled Data) เพื่อหาโครงสร้างที่ซ่อน อยู่	เรียนรู้จาก "ประสบการณ์" (Experience) ที่สร้าง ขึ้นใหม่ตลอดเวลาผ่าน การกระทำ (Action)
ลักษณะการป้อนกลับ (FEEDBACK)	Instant Feedback: รู้ ทันทีว่าทำถูกหรือผิด จาก Label	No Explicit Feedback: ไม่มีถูกผิด มีแต่ความเหมาะสมของ การจัดกลุ่ม	Delayed Reward: รางวัลอาจมาช้า เช่น เดินหมากรุกแต่ยังไม่รู้ ผล ต้องรอจบเกมถึงรู้ว่า ดีหรือไม่
เป้าหมาย (OBJECTIVE)	สร้างฟังก์ชันเพื่อทำนาย (Prediction) หรือ จำแนก (Classification) ให้ แม่นยำที่สุด	ค้นหาแพทเทิร์น (Pattern Discovery) หรือลดมิติข้อมูล (Dimensionality Reduction)	ค้นหา นโยบาย (Policy) หรือลำดับการ ตัดสินใจ (Sequence of Decisions) ที่ดีที่สุด
ตัวอย่าง (EXAMPLE)	การจดจำภาพแมว/หมา , การพยากรณ์ราคา บ้าน	การจัดกลุ่มลูกค้า (Customer Segmentation), การ แนะนำสินค้า	การเล่นเกม (Chess, Go), หุ่นยนต์เดิน, รถยนต์ไร้คนขับ
ความท้าทายหลัก (CORE CHALLENGE)	การหาข้อมูลที่มี คุณภาพ (Garbage In, Garbage Out)	การตีความผลลัพธ์ (Interpretation)	การสมดุลระหว่างการ สำรวจและการกอบโกย (Exploration vs. Exploitation)

1.3. องค์ประกอบหลักของระบบ (The RL Ecosystem)

ระบบ RL ประกอบด้วยองค์ประกอบ 5 ส่วนที่หมุนเวียนเป็นวัฏจักร (Loop) อย่างต่อเนื่อง :

1.3.1. Agent (ตัวแทน): เปรียบเสมือนสมองของผู้เรียนรู้ เป็นผู้ตัดสินใจ (Decision Maker) ในระบบ Agent จะไม่มีความรู้เกี่ยวกับโลกภายนอกมาก่อน แต่จะค่อยๆ สร้างองค์ความรู้ผ่านการลองผิดลองถูก (Trial and Error)

1.3.2. Environment (สภาพแวดล้อม): คือโลกที่ Agent อาศัยอยู่ มีกฎเกณฑ์ทางฟิสิกส์ หรือกติกาที่แน่นอน (เช่น กระดาน Tic-Tac-Toe) Environment จะตอบสนองต่อการกระทำของ Agent เสมอ

1.3.3. State (สถานะ - S_t): ภาพรวมของสถานการณ์ ณ เวลาปัจจุบัน t ที่ Agent รับรู้ได้ เช่น ตำแหน่งของตัวหมากบนกระดาน หรือพิกัดของหุ่นยนต์

1.3.4. Action (การกระทำ - A_t): เซตของทางเลือกทั้งหมดที่ Agent สามารถทำได้ในสถานะนั้นๆ (Action Space) เช่น การเลือกช่องเดิน การเลี้ยงซ้าย/ขวา

1.3.5. Reward (รางวัล - R_t): สัญญาณตัวเลข (Scalar Value) ที่ Environment ส่งกลับมาเพื่อบอกว่า "สิ่งที่ทำไปเมื่อครั้ง นี้ ดีแค่ไหน" รางวัลอาจเป็นบวก (ให้รางวัล) หรือลบ (ลงโทษ) ก็ได้

2. ทฤษฎีคณิตศาสตร์: กระบวนการตัดสินใจแบบมาร์คอฟ (Markov Decision Process: MDP)

เพื่อให้การเรียนรู้ของ Agent เป็นไปอย่างมีระบบ เราต้องตีกรอบปัญหาให้อยู่ในรูปแบบทางคณิตศาสตร์ที่เรียกว่า Markov Decision Process (MDP)

2.1. สมบัติของมาร์คอฟ (The Markov Property)

หัวใจของ MDP คือสมมติฐานที่เรียกว่า Markov Property ซึ่งกล่าวว่า:

"อนาคตขึ้นอยู่กับปัจจุบันเท่านั้น ไม่ขึ้นอยู่กับอดีต"

("The future is independent of the past given the present.")

ในเชิงคณิตศาสตร์ ความน่าจะเป็นที่จะเกิดสถานะใหม่ S_{t+1} และได้รับรางวัล R_{t+1} ขึ้นอยู่กับสถานะปัจจุบัน S_t และการกระทำ A_t เท่านั้น:

$$P(S_{t+1}, R_{t+1} | S_t, A_t)$$

โดยไม่จำเป็นต้องรู้ประวัติการเดิน ($S_{t-1}, S_{t-2}, \dots$) ย้อนหลัง นี่เป็นข้อสมมติที่สำคัญมากที่ทำให้อัลกอริทึม RL สามารถคำนวณได้จริงโดยไม่ต้องเก็บหน่วยความจำมหาศาล ในกรณีของเกม Tic-Tac-Toe กระดานปัจจุบันบอกทุกอย่างที่เราต้องรู้เพื่อตัดสินใจตาถัดไป โดยไม่ต้องย้อนดูว่าก่อนหน้านี้ใครเดินช่องไหนก่อนหลัง

2.2. องค์ประกอบของ MDP Tuple (S, A, P, R, γ)

MDP สามารถเขียนแทนได้ด้วย Tuple 5 ตัว ดังนี้ :

2.2.1. S (State Space): เซตของสถานะทั้งหมดที่เป็นไปได้

2.2.2. A (Action Space): เซตของการกระทำทั้งหมดที่เป็นไปได้

2.2.3. P (Transition Probability): ความน่าจะเป็นที่จะเปลี่ยนไปสู่สถานะใหม่

2.2.4. R (Reward Function): ฟังก์ชันกำหนดรางวัล

2.2.5. γ (Discount Factor): ตัวประกอบการลดทอนค่าเงินในอนาคต

3. สมการเบลล์แมน (The Bellman Equation): เชื่อมทิศสู่ความสำเร็จ

หาก Agent ต้องการชนะเกม มันต้องรู้วิธีประเมินมูลค่าของสถานะ (State Value) ว่าสถานะที่ยืนอยู่ "ดี" หรือ "แย่" แค่ไหน ซึ่งไม่ใช่เรื่องง่ายเพราะรางวัล (การชนะ) อยู่ในอนาคตที่ยังมาไม่ถึง Richard Bellman นักคณิตศาสตร์อัจฉริยะได้คิดค้นสมการในปี 1957 เพื่อแก้ปัญหานี้

3.1. แนวคิดเรื่องมูลค่า (Value) และผลตอบแทน (Return)

เป้าหมายของ Agent คือการหา ผลตอบแทนรวม (Return - G_t) ซึ่งคือผลรวมของรางวัลตั้งแต่วันที่นี้ไปจนจบเกม:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_t$$

แต่ในความเป็นจริง "รางวัลในปัจจุบันย่อมมีค่ามากกว่ารางวัลในอนาคต" (เนื่องจากความไม่แน่นอนของอนาคต) เราจึงต้องคูณด้วย Discount Factor (γ) ที่มีค่าระหว่าง $0 \leq \gamma \leq 1$:

$$G_t = R_{t+1} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

3.1.1. ถ้า $\gamma = 0$: Agent จะเป็นคน "สายตาสั้น" (Myopic) สนใจแต่ความสุขเฉพาะหน้า

3.1.2. ถ้า $\gamma = 1$: Agent จะเป็นคน "มองการณ์ไกล" (Farsighted) ให้ความสำคัญกับอนาคตเท่ากับปัจจุบัน

3.2. โครงสร้างแบบเรียกซ้ำ (Recursive Definition)

ความมหัศจรรย์ของสมการเบลล์แมนคือการเปลี่ยนสมการผลรวมอนันต์ให้กลายเป็นสมการความสัมพันธ์ระหว่าง "ปัจจุบัน" และ "อนาคตเพียงก้าวเดียว"

$$V(s) = \max_a (R + \gamma V(s'))$$
 ให้ง่าย : "มูลค่าของจุดที่เราอยู่นอยู่ ($V(s)$) เท่ากับรางวัลที่เราจะได้เดี๋ยวนี้ (R) บวกกับ มูลค่าที่ดีที่สุดของจุดถัดไปที่เราจะไปถึง ($V(s')$) ที่ถูกลดทอนลงเล็กน้อย (γ)"

4. อัลกอริทึม Q-Learning: การเรียนรู้โดยไม่ต้องรู้กฎโลก

ใน MDP แบบดั้งเดิม เราต้องรู้ความน่าจะเป็นของการเปลี่ยนสถานะ (P) ล่วงหน้า (Model-Based) แต่ในโลกความเป็นจริง เรามักไม่รู้กฎของโลก เราจึงใช้วิธี Model-Free RL และอัลกอริทึมที่ทรงพลังที่สุดคือ Q-Learning

4.1. Q-Table: สมุดจดความรู้

Q-Learning ไม่ได้เก็บแค่ค่าของสถานะ ($V(s)$) แต่เก็บค่าของคู่สถานะและการกระทำ ($Q(s, a)$) หรือ Action-Value ซึ่งบอกว่า "ในสถานะ s นี้ ถ้าทำ a แล้วจะดีแค่ไหน"

โครงสร้างข้อมูลที่ใช้เก็บคือ Q-Table (ตารางคิว):

4.1.1. แถว (Rows): แทนสถานะทั้งหมดที่เป็นไปได้ (All possible states)

4.1.2. คอลัมน์ (Columns): แทนการกระทำทั้งหมดที่เป็นไปได้ (All possible actions)

4.1.3. เซลล์ (Cells): เก็บค่า Q-Value ที่เริ่มต้นจาก 0 และจะถูกอัปเดตให้แม่นยำขึ้นเรื่อยๆ

4.2. กฎการอัปเดตค่า Q (Q-Learning Update Rule)

หัวใจสำคัญของ Code คือสมการการเรียนรู้ (Learning Rule) ซึ่งเป็นการปรับปรุงค่าความรู้เดิมด้วยความรู้ใหม่ :

$$Q_{new}(s, a) \leftarrow (1 - \alpha) \cdot \underbrace{Q_{old}(s, a)}_{\text{ความรู้เดิม}} + \alpha \cdot \underbrace{R + \gamma \cdot \max_{a'} Q(s', a')}_{\text{ความรู้ใหม่ (Target)}}$$

องค์ประกอบของสมการนี้มีความหมายลึกซึ้ง:

4.2.1. $Q_{old}(s, a)$: คือสิ่งที่ Agent เชื่อในปัจจุบัน

4.2.2. $R + \gamma \cdot \max_{a'} Q(s', a')$: คือความเป็นจริงที่ Agent เพิ่งค้นพบ (รางวัลที่ได้จริง + การคาดการณ์อนาคตที่ดีที่สุดจากจุดใหม่) หรือเรียกว่า TD Target

4.2.3. α (Alpha - Learning Rate): คืออัตราการเปิดรับข้อมูลใหม่ (0-1) ถ้าค่าสูง Agent จะเปลี่ยนความคิดเร็ว แต่ถ้าต่ำ Agent จะค่อยๆ เรียนรู้

4.2.4. Temporal Difference (TD) Error: คือส่วนต่างระหว่าง "ความจริงใหม่" กับ "ความเชื่อเดิม" Agent จะพยายามลด Error นี้ให้เป็น 0

4.3. On-Policy vs. Off-Policy

Q-Learning ถูกจัดเป็นอัลกอริทึมแบบ Off-Policy

4.3.1. ความหมาย: ในขณะที่เรียนรู้ Agent อาจเดินสะเปะสะปะเพื่อสำรวจ (Exploration Policy) แต่เวลาอัปเดตสมการ มันจะใช้สมมติฐานว่าในอนาคตมันจะเดินอย่างฉลาดที่สุด ($\max Q$) เสมอ (Target Policy)

4.3.2. ข้อดี: ทำให้ Agent เรียนรู้ "ยุทธวิธีที่ดีที่สุด" (Optimal Policy) ได้ แม้ว่าในขณะที่ฝึกซ้อมมันจะลองผิดลองถูกทำเรื่องผิดพลาดบ้างก็ตาม

5. ยุทธวิธีการสำรวจและการกอบโกย (Exploration vs. Exploitation Dilemma)

ปัญหาคลาสสิกที่สุดของ RL คือการตัดสินใจว่าจะ "ใช้ความรู้เดิมหากิน" (Exploitation) หรือ "เสี่ยงดวงหาทางใหม่" (Exploration)

5.1. ปัญญาของความลังเล (The Dilemma)

5.1.1. Exploitation (การกอบโกย): เลือก Action ที่มีค่า Q สูงสุดในตารางเสมอ เหมือนการเข้าร้านอาหารเดิมที่รู้ว่าอร่อยแน่ๆ ข้อดีคือได้คะแนนเยอะตอนนี้ ข้อเสียคืออาจพลาดร้านที่อร่อยกว่า (ติดอยู่ใน Local Optimum)

5.1.2. Exploration (การสำรวจ): เลือก Action แบบสุ่ม หรือเลือกอันที่ไม่เคยทำ เหมือนการลองสุ่มเข้าร้านใหม่ ข้อดีคือมีโอกาสเจอทางลัดสู่ชัยชนะที่ดีกว่า ข้อเสียคือเสี่ยงที่จะเจอผลลัพธ์ที่แย่

5.2. Epsilon-Greedy Strategy (ϵ -Greedy)

วิธีแก้ปัญหาที่ใช้ใน Tic-Tac-Toe Project นี้คือกลยุทธ์ Epsilon-Greedy :

5.2.1. กำหนดค่า ϵ (Epsilon) เป็นความน่าจะเป็นในการสุ่ม (เช่น 0.1 หรือ 10%)

5.2.2. สุ่มตัวเลข r ระหว่าง 0-1

5.2.2.1. ถ้า $r < \epsilon$: Explore (สุ่มเดินมั่ว)

5.2.2.2. ถ้า $r \geq \epsilon$: Exploit (เดินตาม Q-Table ที่ค่ามากที่สุด)

5.3. Adaptive Epsilon (Epsilon Decay)

เพื่อให้การเรียนรู้มีประสิทธิภาพสูงสุด เรามักใช้เทคนิค Decay คือ เริ่มต้นด้วย $\epsilon = 1.0$ (สุ่ม 100% เพื่อเรียนรู้โลกให้กว้าง) แล้วค่อยๆ ลดค่าลงเรื่อย ๆ ตามจำนวนรอบที่ฝึก (เช่น คูณด้วย 0.999 ทุกรอบ) จนเหลือค่าต่ำ ๆ (เช่น 0.01) เพื่อให้ Agent เริ่มเอาจริงในช่วงท้าย

6. กรณีศึกษา: การสร้าง AI เล่นเกม Tic-Tac-Toe (Project Deep Dive)

6.1. การออกแบบพื้นที่สถานะ (State Space Engineering)

Tic-Tac-Toe เป็นเกมกระดานขนาด 3x3 ซึ่งดูเรียบง่ายแต่มีความซับซ้อนซ่อนอยู่

6.1.1. จำนวนสถานะทางทฤษฎี: ช่อง 9 ช่อง แต่ละช่องเป็นไปได้ 3 แบบ (ว่าง, X, O) ดังนั้นจำนวนรูปแบบคือ $3^9 = 19,683$ แบบ

6.1.2. จำนวนสถานะจริง (Reachable States): เนื่องจากกติกาเกม (เช่น X และ O จำนวนต้องต่างกันไม่เกิน 1, เกมจบเมื่อมีผู้ชนะ) ทำให้สถานะที่เกิดขึ้นจริงมีเพียงประมาณ 5,478 สถานะเท่านั้น นี่คือเหตุผลว่าทำไมเราจึงใช้ Tabular Q-Learning (ใช้ตารางเก็บ) ได้ เพราะหน่วยความจำที่ใช้ไม่มากเกินไป

6.1.3. การแทนข้อมูล (Representation): ใน Python Code เราไม่สามารถใช้อาร์เรย์ 2 มิติเป็น Key ของ Dictionary ได้ จึงต้องแปลงกระดานให้เป็น String หรือ Tuple (Hashable) เช่น `str([1, 0, -1, 0, 0, ...])` เพื่อใช้ระบุ State ใน Q-Table

6.2. การออกแบบระบบรางวัล (Reward Shaping)

ระบบรางวัลคือวิธีที่มนุษย์ใช้สื่อสารเป้าหมายกับ AI หากออกแบบผิด AI จะมีพฤติกรรมที่ไม่พึงประสงค์ :

6.2.1. Win (ชนะ): ได้รับ +1 (แรงจูงใจบวกสูงสุด)

6.2.2. Lose (แพ้): ได้รับ -1 (แรงจูงใจลบ เพื่อให้หลีกเลี่ยง)

6.2.3. Draw (เสมอ): ได้รับ 0 (หรือบางทฤษฎีให้คะแนนบวกเล็กน้อย เช่น 0.1-0.5 หากต้องการให้ AI เน้นการไม่แพ้มากกว่าการเสี่ยงชนะ)

6.2.4. Move (เดินหมากทั่วไป): ได้รับ 0 (Delayed Reward) เพื่อให้ AI ไม่สนใจรางวัลระหว่างทาง แต่มุ่งเน้นผลลัพธ์ท้ายเกมเท่านั้น

6.3. เทคนิคขั้นสูง: การลดรูปสมมาตร (Symmetry Reduction)

เพื่อเพิ่มประสิทธิภาพการเรียนรู้ เราสามารถใช้ความรู้ทางเรขาคณิตมาช่วยลดขนาด State Space ได้ :

6.3.1. Rotation Invariance: กระจกานที่หมุน 90, 180, 270 องศา ถือเป็นสถานะเดียวกันเชิงกลยุทธ์

6.3.2. Reflection Invariance: กระจกานที่พลิกซ้าย-ขวา หรือบน-ล่าง ก็ถือเป็นสถานะเดียวกัน

6.3.3. Canonical State: เทคนิคนี้จะแปลงทุกสถานะให้เป็น "รูปแบบมาตรฐาน" (Canonical Form) ก่อนเก็บลง Q-Table เช่น หากเจอสถานะที่หมุนมา ก็ให้หมุนกลับไปเทียบกับฐานข้อมูลเดิม ทำให้ AI เรียนรู้ 1 สถานะ แต่นำไปใช้ได้กับอีก 7 สถานะที่สมมาตรกัน ช่วยให้การเรียนรู้เร็วขึ้นถึง ~8 เท่า

6.4. การเรียนรู้แบบวนลูป (Human-in-the-loop Implementation)

6.4.1. History Tracking: ระหว่างเกม ระบบจะบันทึก History ของการเดิน (State, Action) ทั้งหมดไว้

6.4.2. Backward Propagation: เมื่อรู้ผลแพ้ชนะ ระบบจะนำรางวัลสุดท้าย (+1/-1) มาคำนวณย้อนกลับเพื่ออัปเดตค่า Q ของทุกท่าที่เดินมาในเกมนั้น ทำให้ AI "ฉลาดขึ้น" ทันทีหลังจากเล่นกับเรา

7. การประเมินผลและการปรับจูน (Evaluation & Tuning)

ในการทำโปรเจกต์ RL การวัดผลด้วย Accuracy เพียงอย่างเดียวไม่เพียงพอ เราต้องใช้ตัวชี้วัด (Metrics) ที่สะท้อนพฤติกรรมของ Agent ได้ดีกว่า

7.1. ตัวชี้วัดประสิทธิภาพ (Performance Metrics)

7.1.1. กราฟรางวัลสะสม (Cumulative Reward Curve): พล็อตกราฟผลรวมรางวัลเฉลี่ยในแต่ละช่วง Episode กราฟควรมีแนวโน้มสูงขึ้นและลู่เข้าสู่ค่าสูงสุด

7.1.2. อัตราการชนะ (Win Rate): ทดสอบโดยให้ Agent สู้กับ Random Bot หรือ Expert Bot จำนวน 100-1,000 เกม แล้วดูเปอร์เซ็นต์การชนะ การเสมอ และการแพ้

7.1.3. การลู่เข้า (Convergence): ตรวจสอบว่าค่าใน Q-Table หยุดเปลี่ยนแปลงอย่างรุนแรงหรือไม่ (Stable Q-Values) ซึ่งแสดงว่า Agent ได้เรียนรู้จนถึงจุดอิ่มตัวแล้ว

7.2. ผลกระทบของไฮเปอร์พารามิเตอร์ (Hyperparameter Effects)

ตารางนี้สรุปผลของการปรับค่าพารามิเตอร์ต่างๆ เพื่อเป็นแนวทางให้นักศึกษานำไปทดลอง (Experimentation) :

พารามิเตอร์ (PARAMETER)	ค่าต่ำ (LOW VALUE)	ค่าสูง (HIGH VALUE)	ผลต่อ TIC-TAC-TOE AI
LEARNING RATE (α)	เรียนรู้ช้ามาก ต้องเล่น หลายแสนตา	เรียนรู้เร็วแต่ไม่เสถียร ค่า Q แกว่ง (Oscillate)	ควรใช้ค่ากลางๆ เช่น 0.1 - 0.2 เพื่อความ สมดุล
DISCOUNT FACTOR (γ)	สนใจแต่รางวัลเฉพาะ หน้า (Myopic)	มองการณ์ไกล (Farsighted) สนใจผล จบเกม	ควรใช้ค่าสูง 0.9 - 0.99 เพราะผลแพ้ชนะอยู่ที่ ปลายทางเสมอ
EPSILON DECAY	ลดการสุ่มเร็วเกินไป อาจไม่เจอท่ายาก	ลดช้าเกินไป เสียเวลาฝึก นานกว่าจะเก่ง	ควรตั้งให้ลดลงเรื่อยๆ จนเหลือขั้นต่ำ (Min Epsilon) ประมาณ 0.01 - 0.1

อ้างอิง

Adeniran, A. (2020). *Reinforcement learning for TicTacToe AI*. Kaggle.

<https://www.kaggle.com/code/adedapoadeniran/reinforcement-learning-for-tictactoe-ai>

Aitude. (n.d.). *Supervised vs unsupervised vs reinforcement*.

<https://www.aitude.com/supervised-vs-unsupervised-vs-reinforcement/>

Ballal, A. (2020, May 19). Reinforcement learning from scratch part 1: Tabular Q-learning.

Dev.to. <https://dev.to/akshayballal/reinforcement-learning-from-scratch-part-1-tabular-q-learning-22k3>

Bellman equation. (2024, June 10). In *Wikipedia*.

https://en.wikipedia.org/wiki/Bellman_equation

Bernard, E. (2018, December 8). Introduction to reinforcement learning: Chapter 1. *Medium*.

<https://medium.com/data-science/introduction-to-reinforcement-learning-chapter-1-fc8a196a09e8>

Bugfree.ai. (n.d.). *Evaluating RL agents: Metrics & benchmarks*. <https://bugfree.ai/knowledge-hub/evaluating-rl-agents-metrics-benchmarks>

Danushidk507. (2023, June 10). Bellman equation in reinforcement learning. *Medium*.

<https://medium.com/@danushidk507/bellman-equation-in-reinforcement-learning-16c677538baa>

DataCamp. (2023). *Bellman equation in reinforcement learning*.

<https://www.datacamp.com/tutorial/bellman-equation-reinforcement-learning>

DataCamp. (2023). *Introduction to Q-Learning beginner tutorial*.

<https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial>

DeepLearning.TV. (2017, August 30). *Reinforcement learning - Ep. 30 (Deep Learning SIMPLIFIED)* [Video]. YouTube.

<https://www.youtube.com/watch?v=820w7jGXYsc>

DevInterview. (n.d.). *Q-learning interview questions*. GitHub.

<https://github.com/Devinterview-io/q-learning-interview-questions>

Efroni, Y., & Mannor, S. (2022). *Reinforcement learning with epsilon-greedy exploration*. arXiv.

<https://arxiv.org/abs/2212.12252>

FreeCodeCamp. (2020, June 4). *How to build an adaptive Tic-Tac-Toe AI with reinforcement learning in JavaScript*. <https://www.freecodecamp.org/news/how-to-build-an-adaptive-tic-tac-toe-ai-with-reinforcement-learning-in-javascript/>

GeeksforGeeks. (2024, January 10). *On-policy vs off-policy methods reinforcement learning*. <https://www.geeksforgeeks.org/machine-learning/on-policy-vs-off-policy-methods-reinforcement-learning/>

Goudarzi, R. (2023, May 11). The magic of AI: The Bellman equation. *Medium*. <https://medium.com/@rohamgoudarz/the-magic-of-ai-the-bellman-equation-51bddac5fd65>

IFOA Data Science. (n.d.). *Comparison of supervised, unsupervised and reinforcement learning*. <https://ifoodatascienceresearch.github.io/tutorial/comparison/>

Jang, B., Kim, M., Harerimana, G., & Kim, J. W. (2019). Q-Learning algorithms: A comprehensive classification and applications. *IEEE Access*, 7, 133653–133667. <https://yonsei.elsevierpure.com/en/publications/q-learning-algorithms-a-comprehensive-classification-and-applicat>

Kovalchuk, G. (2024, October 24). A beginner's guide to Q-Learning: Understanding with a simple GridWorld example. *Medium*. <https://medium.com/@goldengrisha/a-beginners-guide-to-q-learning-understanding-with-a-simple-gridworld-example-2b6736e7e2c9>

LessWrong. (2019, June 29). *Book review: Reinforcement learning by Sutton and Barto* [Online forum post]. <https://www.lesswrong.com/posts/6zBnNjZfqTNLjP9j5/book-review-reinforcement-learning-by-sutton-and-barto>

Mandeep0405. (2019, April 14). Q-learning: From Bellman equations to model-free learning. *Medium*. <https://medium.com/@mandeep0405/q-learning-from-bellman-equations-to-model-free-learning-31400fe332ea>

Mei, B. (2019). *Reinforcement learning*. Bill Mei. <https://billmei.net/books/reinforcement-learning/>

Milvus. (n.d.). *How do you evaluate the performance of a reinforcement learning agent*. <https://milvus.io/ai-quick-reference/how-do-you-evaluate-the-performance-of-a-reinforcement-learning-agent>

Milvus. (n.d.). *What is exploration versus exploitation in reinforcement learning.*

<https://milvus.io/ai-quick-reference/what-is-exploration-versus-exploitation-in-reinforcement-learning>

Reddit. (2023). *Is the purpose of gamma in Q-learning just to help convergence?* [Online forum post].

https://www.reddit.com/r/reinforcementlearning/comments/10tvdc3/is_the_purpose_of_gamma_in_qlearning_just_to_help/

Reinforcement learning. (2024, June 12). In *Wikipedia*.

https://en.wikipedia.org/wiki/Reinforcement_learning

ResearchGate. (2015). *Explain the term Q-Learning* [Online forum post].

https://www.researchgate.net/post/Explain_the_term_Q-Learning

Rudes, R. (2020, September 8). An introductory reinforcement learning project: Learning Tic-Tac-Toe via self-play tabular Q-learning. *Medium*. <https://medium.com/data-science/an-introductory-reinforcement-learning-project-learning-tic-tac-toe-via-self-play-tabular-b8b845e18fe>

Scribbr. (n.d.). *What is the exploration vs. exploitation trade-off in reinforcement learning?*

<https://www.scribbr.com/frequently-asked-questions/what-is-the-exploration-vs-exploitation-trade-off-in-reinforcement-learning/>

Stack Exchange. (2015). *What is the difference between off-policy and on-policy learning?*

[Online Q&A]. <https://stats.stackexchange.com/questions/184657/what-is-the-difference-between-off-policy-and-on-policy-learning>

Stack Exchange. (2017). *Non-neural network algorithms for large state space in zero sum games* [Online Q&A].

<https://ai.stackexchange.com/questions/19856/non-neural-network-algorithms-for-large-state-space-in-zero-sum-games>

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

<http://incompleteideas.net/book/the-book-2nd.html>

The Informaticists. (2021, August 25). *Learning to play Tic-Tac-Toe via reinforcement*

learning. <https://theinformaticists.wordpress.com/2021/08/25/learning-to-play-tic-tac-toe-via-reinforcement-learning/>

Twice22. (n.d.). *Tic-Tac-Toe*. GitHub Pages. <https://twice22.github.io/tictactoe/>

Yadav, A. (2024, September 20). Tic-Tac-Toe reinforcement learning project. *Medium*.
<https://medium.com/biased-algorithms/tic-tac-toe-reinforcement-learning-project-23f015711f0d>